SOLID Principles

What is SOLID

SOLID is an acronym for five design principles that make object-oriented code more maintainable, flexible, and understandable. These come up specifically in interviews for product-based companies and system design rounds, beyond just basic OOP definitions.

S - Single Responsibility Principle

A class should have only one reason to change — meaning it should do exactly one job. If a class handles both 'saving a file' and 'formatting a report', those are two responsibilities that should be split into two classes. This makes code easier to test and modify without unintended side effects.

O - Open/Closed Principle

Classes should be open for extension but closed for modification. You should be able to add new functionality (e.g. a new shape type) without changing existing, already-tested code — typically achieved through inheritance or interfaces rather than editing existing class logic directly.

L - Liskov Substitution Principle

Objects of a subclass should be usable anywhere the parent class is expected, without breaking the program. If Bird has a fly() method and Penguin extends Bird but can't fly, substituting a Penguin where a Bird is expected breaks the program's assumptions — this is a classic Liskov violation.

I - Interface Segregation Principle

Clients shouldn't be forced to depend on methods they don't use. Instead of one large, do-everything interface, split it into smaller, specific interfaces — a class implementing 'Printer' shouldn't also be forced to implement irrelevant 'Scanner' methods.

D - Dependency Inversion Principle

High-level modules shouldn't depend directly on low-level modules — both should depend on abstractions (interfaces). This decouples code so you can swap implementations (e.g. switching a database) without rewriting the code that uses it.

Why This Matters Beyond Interviews

SOLID isn't just interview trivia — codebases at real companies are judged on these principles during code review. Being able to explain SOLID with an example demonstrates you can write production-quality code, not just solve isolated DSA problems.

Common Interview Questions

- Explain SOLID with a real or hypothetical example for each letter.
- Give an example of code that violates the Single Responsibility Principle and how you'd fix it.

- What design patterns commonly help implement the Open/Closed Principle? (Strategy, Factory)
- Why is favoring composition over inheritance often recommended alongside SOLID?